

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen

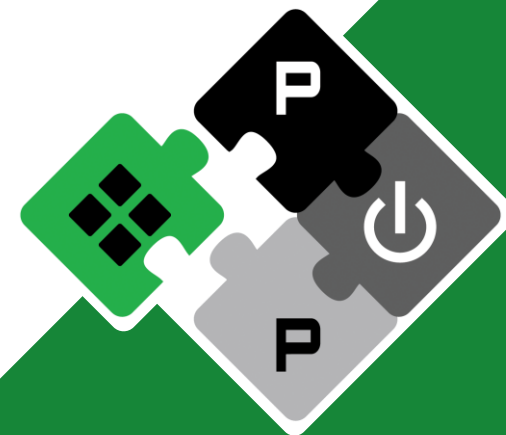
zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli
Luca Benini

avanelli@iis.ee.ethz.ch
lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



@pulp_platform



pulp-platform.org



youtube.com/pulp_platform





- **Q: What is the actual L1 slice memory size?**
 - A: It is 256KiB per Tile
- **Q: Which process was used for those results, the 7nm one? Which process is the 7nm one exactly?**
 - TSMC7 FinFET. Tables below for SRAM size and count. Table 1 is for original design, Table 2 for folded cache
 - Table 1: Original cache design

Purpose/Role	Config	Cnt per unit/controller	Cnt per Tile	Cnt per Group
L1 D\$ Data	128x128b Need 128x512b	$4*2*4 = 32$ 4 way, 2 pseudo way, 4 128b to form 512b	$4*32 = 128$	$4*128 = 512$
L1 D\$ Tag	128x128b use 92b	$4*2 = 8$	$4*8 = 32$	$4*32 = 128$
L1 I\$ Data	128x128b	4	4	$4*4 = 16$
L1 I\$ Tag	128x38b use 24b	4	4	$4*4 = 16$
Stack	256x32b	1	$4*1 = 4$	$4*4 = 16$





- **Q: What is the actual L1 slice memory size?**
 - A: It is 256KiB per Tile
- **Q: Which process was used for those results, the 7nm one? Which process is the 7nm one exactly?**
 - TSMC7 FinFET. Tables below for SRAM size and count. Table 1 is for original design, Table 2 for folded cache
 - Table 2: Folded cache design

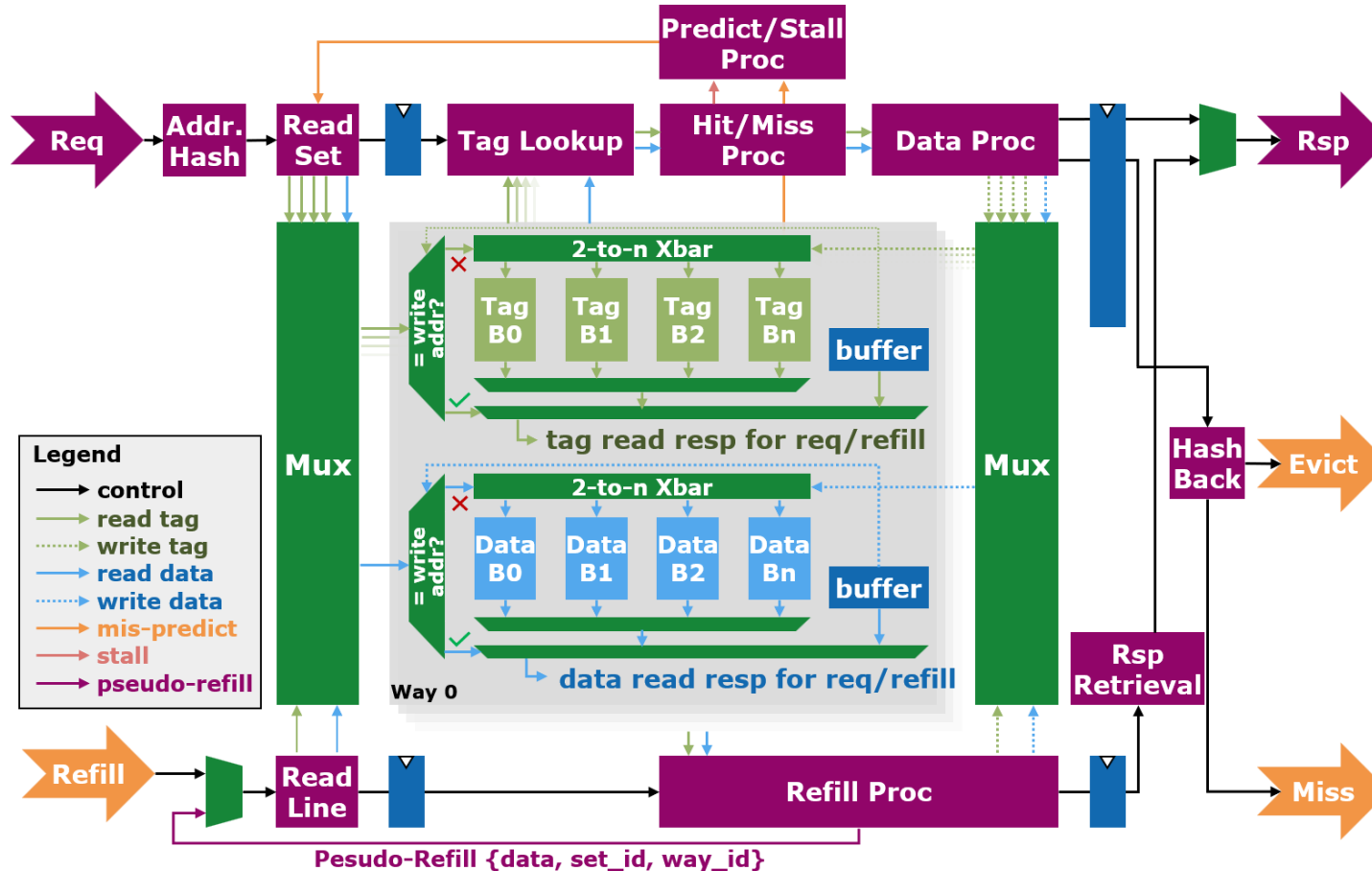
Purpose/Role	Config	Cnt per unit/controller	Cnt per Tile	Cnt per Group
L1 D\$ Data	512x128b	$4 * 2 = 8$ 4 way, 2 pseudo way,	$4 * 8 = 32$	$4 * 32 = 128$
L1 D\$ Tag	128x128b use 92b	$4 * 2 = 8$	$4 * 8 = 32$	$4 * 32 = 128$
L1 I\$ Data	128x128b	4	4	$4 * 4 = 16$
L1 I\$ Tag	128x38b use 24b	4	4	$4 * 4 = 16$
Stack	256x32b	1	$4 * 1 = 4$	$4 * 4 = 16$





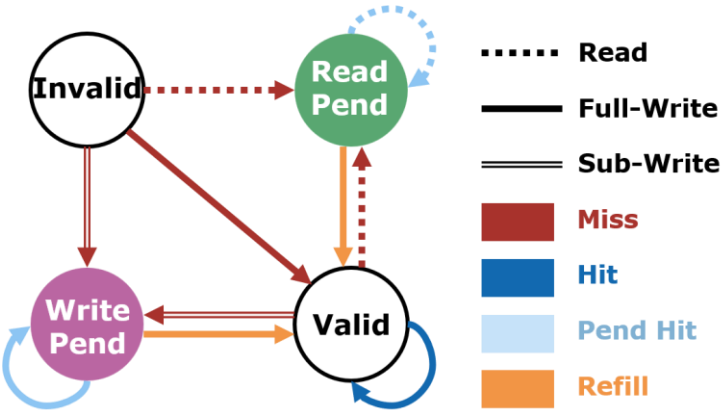
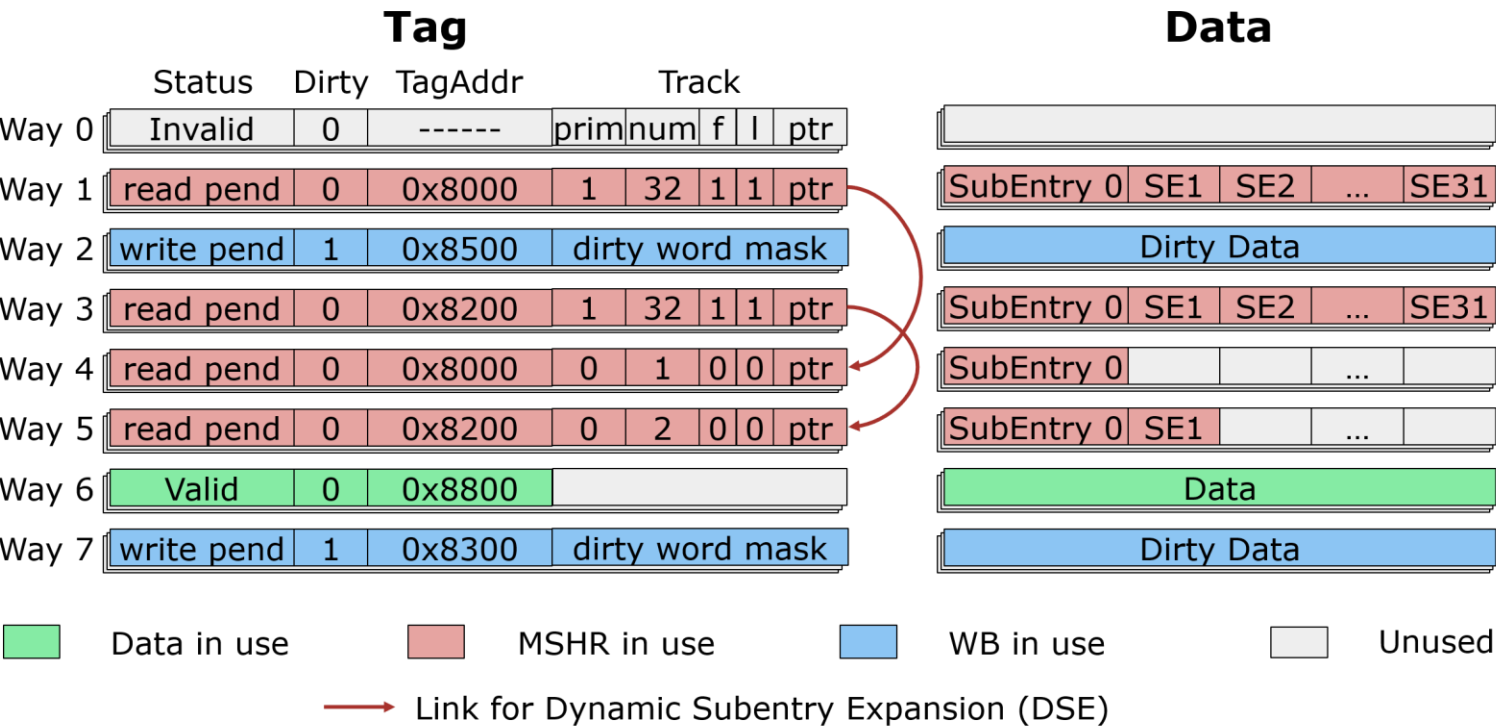
- **Q: More information on Insitu cache**

- A: I don't think we can put the paper on the arxiv now due to double-blind process. The paper results will out on August 20. Here I attached some block diagram of the design





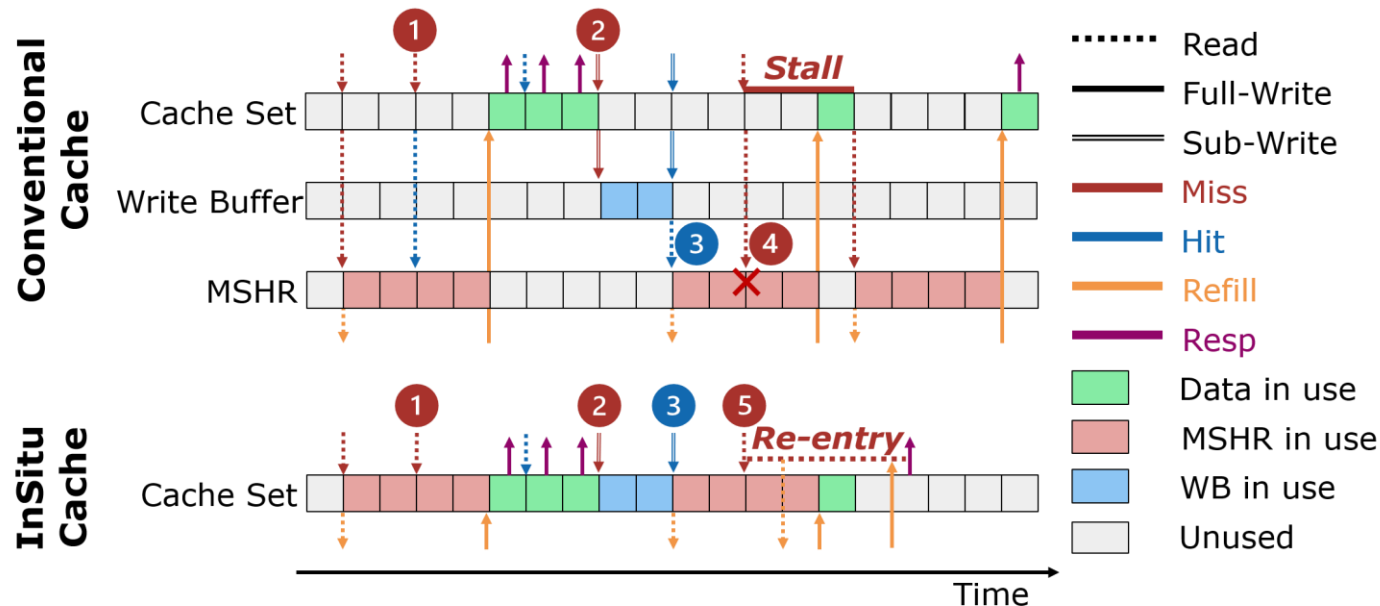
- **Q: More information on Insitu cache**
 - A: I don't think we can put the paper on the arxiv now due to double-blind process. The paper results will out on August 20. Here I attached some block diagram of the design





- **Q: More information on Insitu cache**

- A: I don't think we can put the paper on the arxiv now due to double-blind process. The paper results will out on August 20. Here I attached some block diagram of the design
- A: Caption: Timing diagram comparing resource usage in conventional cache and In-Situ Cache. ① Read-miss MSHR-hit; ② Sub-cacheline write miss, filling write buffer; ③ Sub-cacheline hit in write buffer; ④ Read-miss MSHR-full stall in conventional cache; ⑤ Read-miss MSHR-full, re-entering a different way to avoid stalling in In-Situ Cache.





- **Q: Folded cache block diagram**

- A: See following animations on a 2 way, 2 pseudo way, 256b-cacheline origin design and 128b-cacheline folded design. Core access granularity is 128b, and refill data path is 256b.
- Animation sequence: Miss read block 0 -> trigger MSHR write to block 0, meanwhile read block 2 -> Refill block 0 into way 0
- Assuming LRU policies in both design
- Legends



Write



Read





Way 0 Data Bank

Origin

Pseudo
Way 0
SRAM



BLK0	BLK1
BLK4	BLK5

$2 \times 256b$

Pseudo
Way 1 
SRAM

BLK2	BLK3
BLK6	BLK7

$2 \times 256b$

Way 1 Data Bank

Pseudo
Way 0
SRAM

BLK0	BLK1
BLK4	BLK5

$2 \times 256b$

Pseudo
Way 1
SRAM

BLK2	BLK3
BLK6	BLK7

$2 \times 256b$

Fold

BLK0
BLK1
BLK4
BLK5

$4 \times 128b$

BLK2
BLK3
BLK6
BLK7

$4 \times 128b$

BLK0
BLK1
BLK4
BLK5

$4 \times 128b$

BLK2
BLK3
BLK6
BLK7

$4 \times 128b$





Cycle 0: Miss Read Cacheline 0

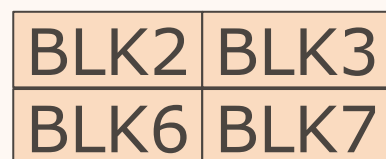
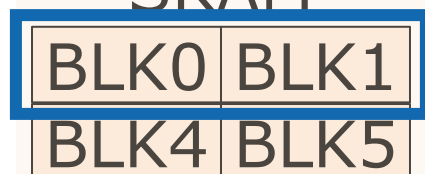
Way 0 Data Bank

Way 1 Data Bank

Origin

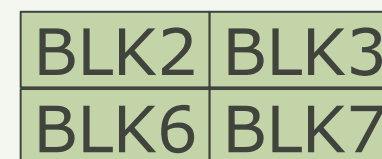
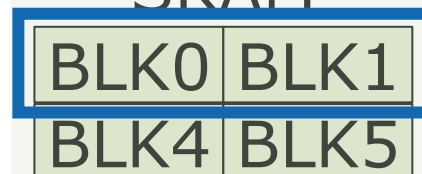
Pseudo
Way 0
SRAM

Pseudo
Way 1
SRAM

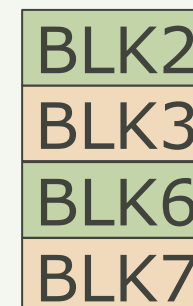
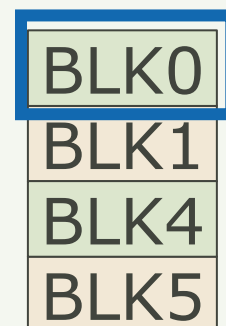
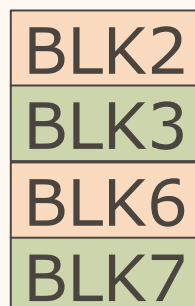
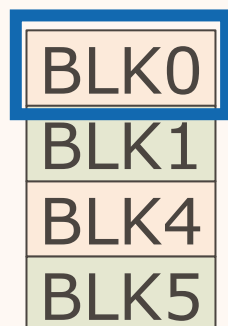


Pseudo
Way 0
SRAM

Pseudo
Way 1
SRAM



Fold





Cycle 1: Update MSHR in Block 0; Read Block 2

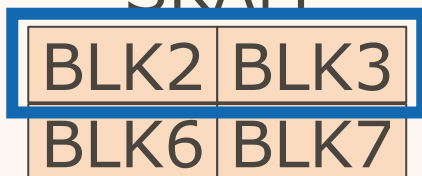
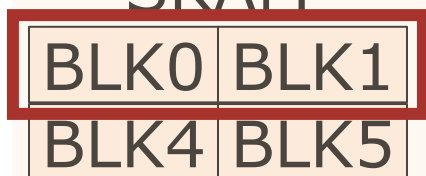
Way 0 Data Bank

Way 1 Data Bank

Origin

Pseudo
Way 0
SRAM

Pseudo
Way 1
SRAM

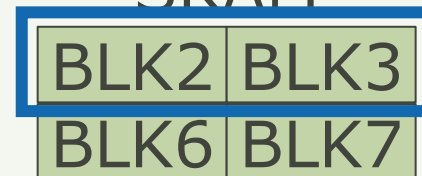
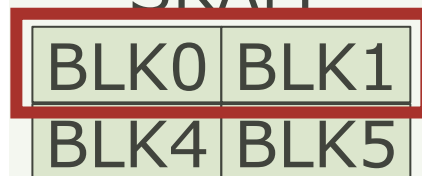


2x256b

2x256b

Pseudo
Way 0
SRAM

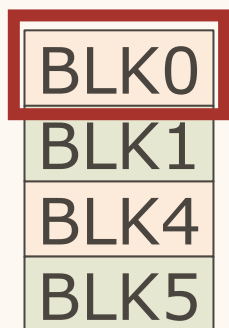
Pseudo
Way 1
SRAM



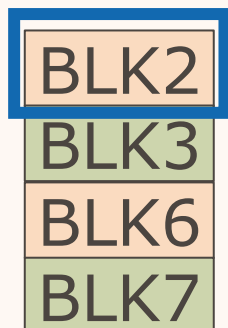
2x256b

2x256b

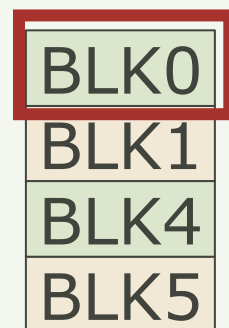
Fold



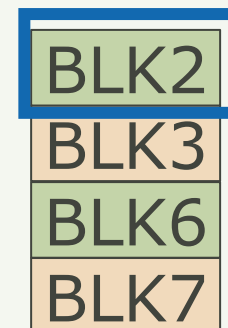
4x128b



4x128b



4x128b



4x128b



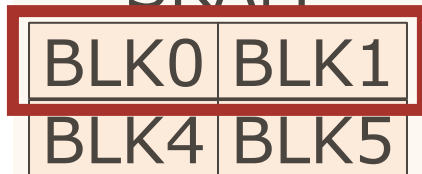
Cycle 2: Refill Cacheline 0 in Way 0



Origin

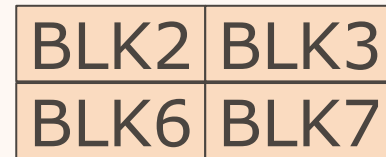
Way 0 Data Bank

Pseudo
Way 0
SRAM



2x256b

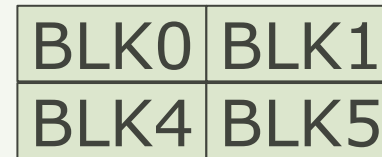
Pseudo
Way 1
SRAM



2x256b

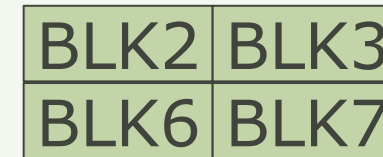
Way 1 Data Bank

Pseudo
Way 0
SRAM



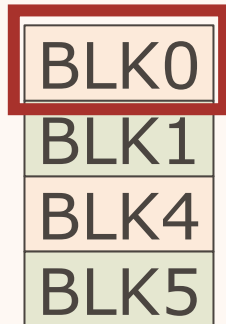
2x256b

Pseudo
Way 1
SRAM

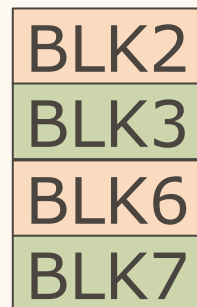


2x256b

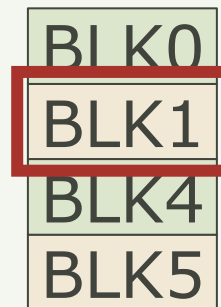
Fold



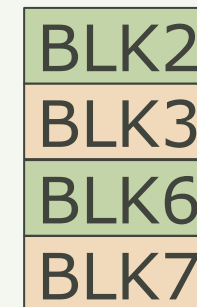
4x128b



4x128b



4x128b



4x128b



Forwarding Buffer



- Q: The main issue was 1 read+ 1 write to the same address in a data or metadata SRAM for a single request, is this correct?
- A: Yes, more details in following slides
- **Q: Explain in some more detail how this issue was also present in the original In-Situ cache, but previously hidden and then made visible by folded cache**
- **A: In short, the Insitu cache pipeline needs to do read-modify-write to both data & meta SRAM for each request, which will take 2 cycles per request.**
 - In the origin design , the coalescer would collect 4x32-bit reqs from spatz per cycle, and issue 1x256-bit req per 2 cycles to the cache ctrl pipeline, and the cache ctrl will read the full 512-bit cache line from data SRAM
 - In folded cache design, as we only read $\frac{1}{4}$ of the cache line from each way, each request to the cache ctrl can only be 128-bit, thus the coalescer needs to send 1 req per cycle to the cache ctrl pipeline.



The Insitu cache SRAM read-modify-write pipeline



- **The read hit flow of insitu cache ctrl**

- Cycle 1: Preread SRAM
 - read meta SRAM (need LRU + dirty + tag)
 - read data SRAM for all ways data
- Cycle 2: FSM + Write SRAM
 - Generate new LRU
 - write meta SRAM (updated LRU)
- Cycle 3: Next preread can issue

For continuous read hit to the same cache line (512b), 2 cycles per request



The Insitu cache SRAM read-modify-write pipeline



- **The write hit flow of insitu cache ctrl**

- Cycle 1: Preread SRAM
 - read meta SRAM (need cache_mask + dirty + LRU + tag)
 - read data SRAM for all ways
- Cycle 2: FSM + Write SRAM
 - generate new_mask byte dirty mask
 - generate new LRU
 - merge new bytes into old line
 - write meta SRAM (updated dirty mask + dirty + LRU)
 - write data SRAM
- Cycle 3: Next preread can issue

For continuous write hit to the same cache line (512b), 2 cycles per request



The Insitu cache SRAM read-modify-write pipeline



- **The cache miss flow of insitu cache ctrl**
 - Cycle 1: Preread SRAM
 - read meta SRAM
 - read data SRAM for all ways
 - Cycle 2: FSM + Write SRAM
 - generate new mshr
 - generate new dirty mask, dirty, LRU
 - write meta SRAM (updated dirty mask + dirty + LRU)
 - write data SRAM (update MSHR)
 - Cycle 3: Next preread can issue

For continuous miss to the same cache line (512b), 2 cycles per request



Previous Design

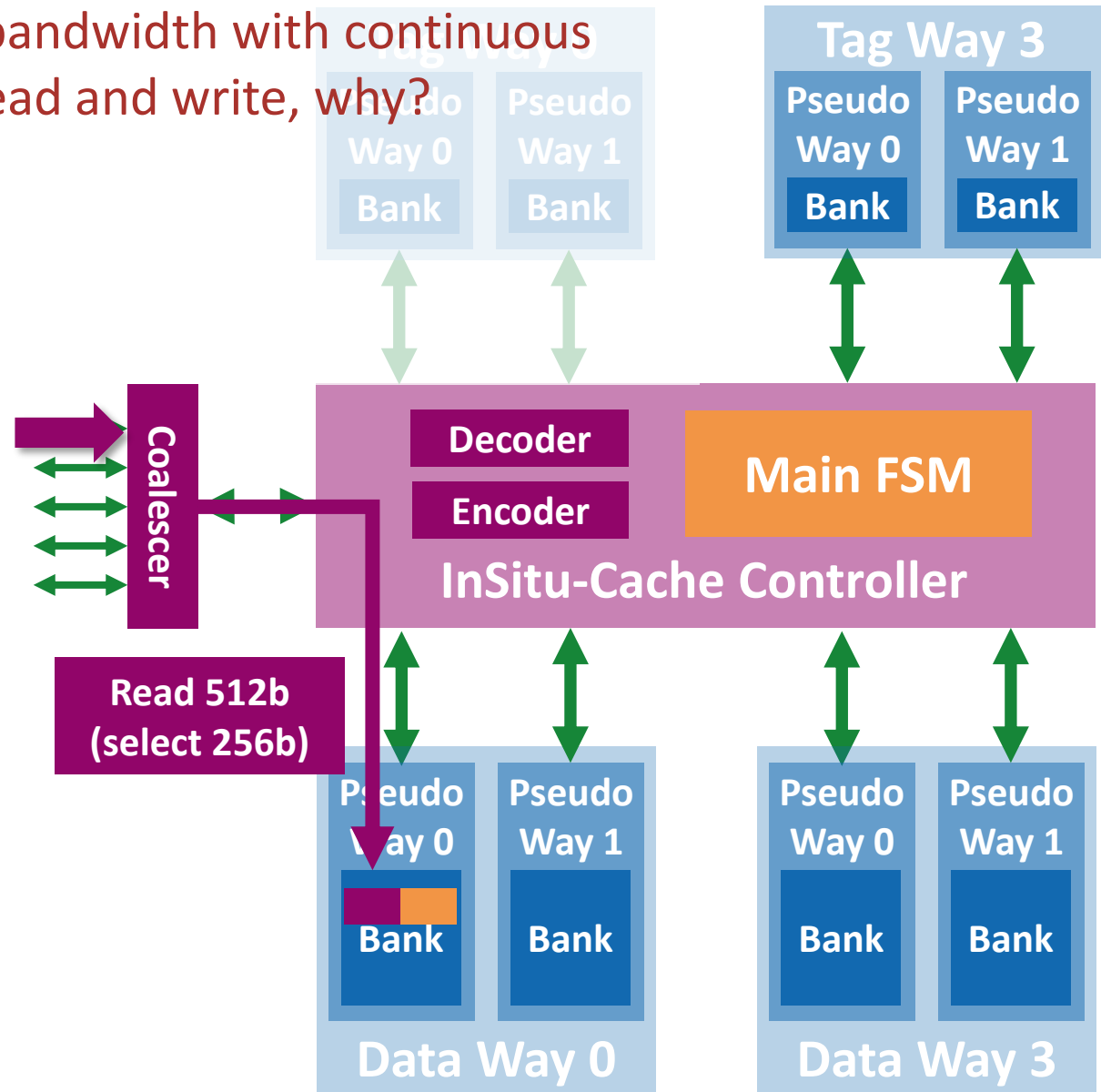
- **Granularity Mismatch**

- Cacheline: 512b, match with DRAM
 - Each access gets a 512b data, select 256b out of it
- Core: 128b (4*32bit)

- **Previous Design**

- Use Coalescer to absorb mismatch
- Every **2 cycles** form a 256b visit
- Read 512b with 2 visits from same entry
- Coalescer needs **2 cycles** to output one **256b** request, which hides the conflict cycle

But the previous design can still sustain the full bandwidth with continuous vector read and write, why?



Previous Design

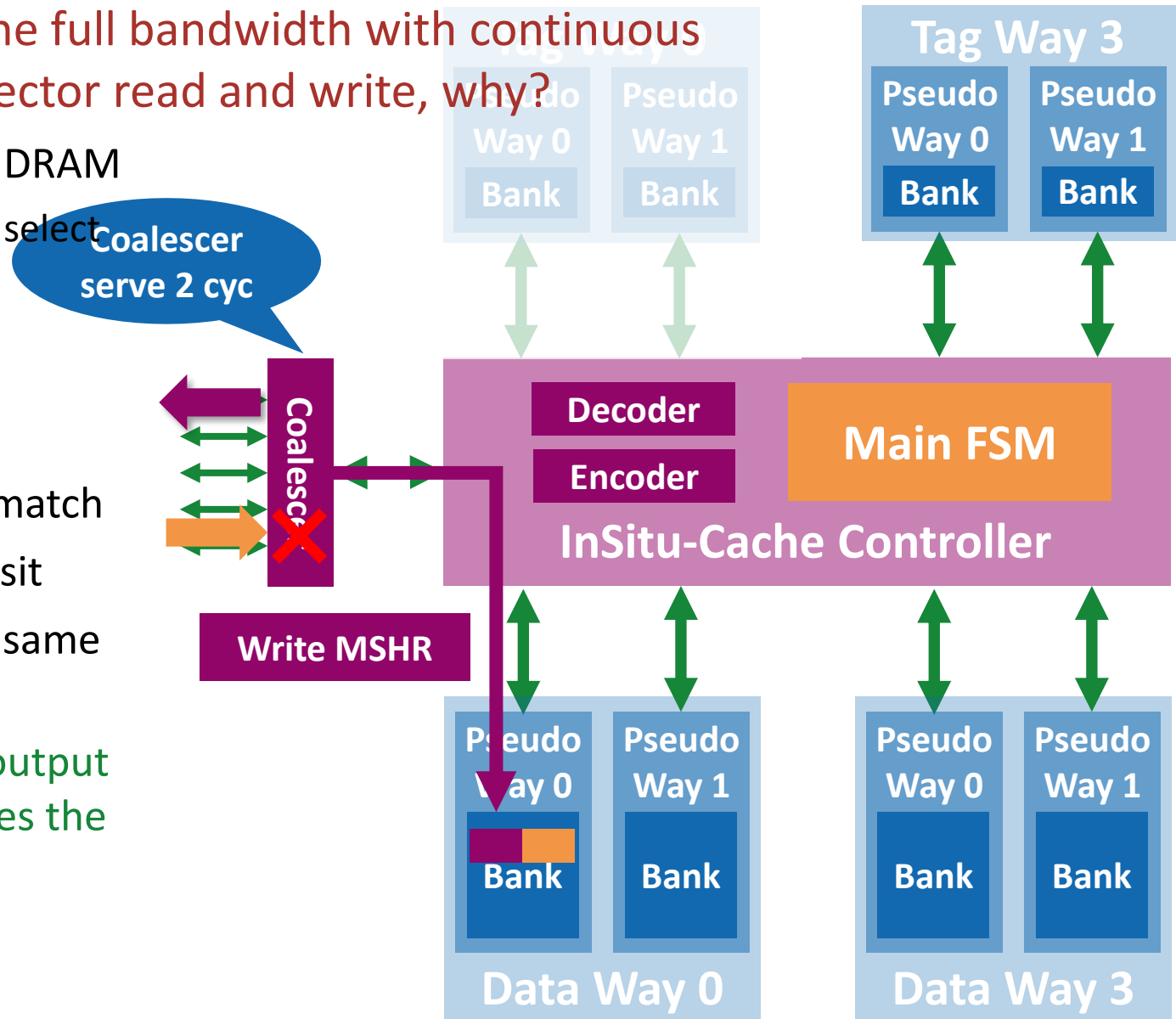
- **Granularity Mismatch**

- Cacheline: 512b, match with DRAM
 - Each access gets a 512b data, select 256b out of it
- Core: 128b (4*32bit)

- **Previous Design**

- Use Coalescer to absorb mismatch
- Every **2 cycles** form a 256b visit
- Read 512b with 2 visits from same entry
- Coalescer needs **2 cycles** to output one **256b** request, which hides the conflict cycle

But the previous design can still sustain the full bandwidth with continuous vector read and write, why?



Previous Design

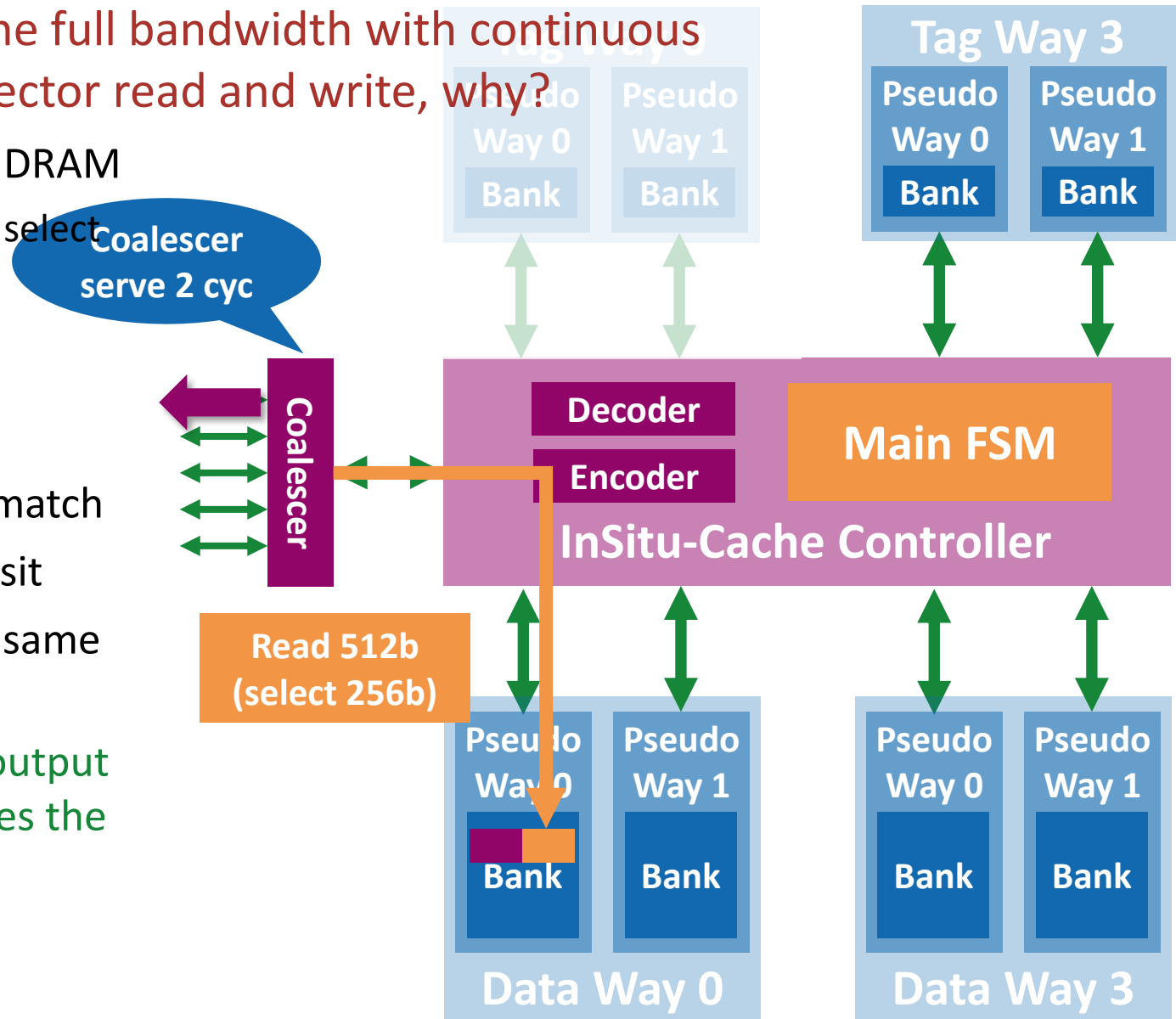
- **Granularity Mismatch**

- Cacheline: 512b, match with DRAM
 - Each access gets a 512b data, select 256b out of it
- Core: 128b (4*32bit)

- **Previous Design**

- Use Coalescer to absorb mismatch
- Every **2 cycles** form a 256b visit
- Read 512b with 2 visits from same entry
- Coalescer needs **2 cycles** to output one **256b** request, which hides the conflict cycle

But the previous design can still sustain the full bandwidth with continuous vector read and write, why?



Folded Cache

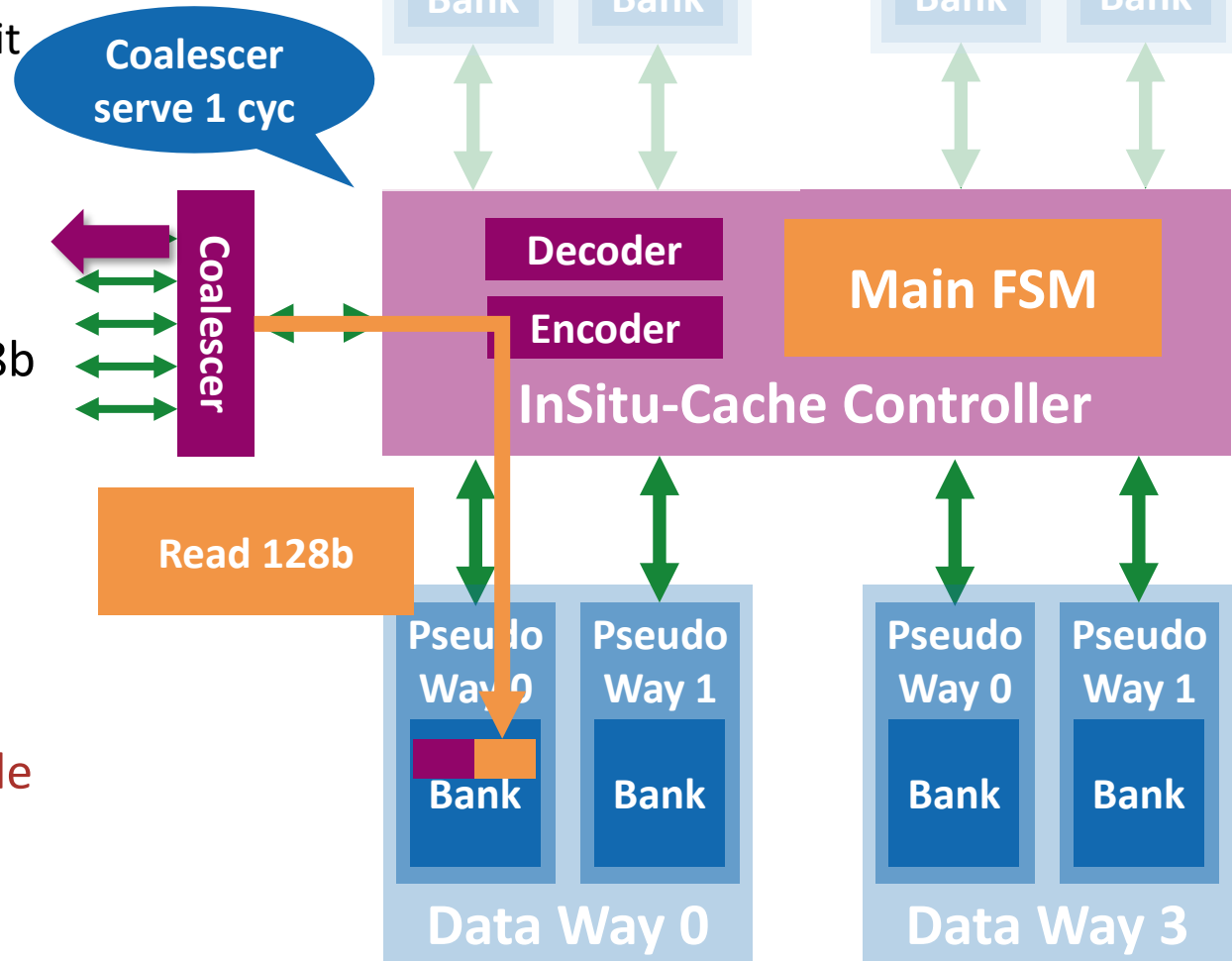
- **Granularity Mismatch**

- Cacheline: 512b, match with DRAM
 - Folded cache: Each access gets 128bit part
- Core: 128b (4*32bit)

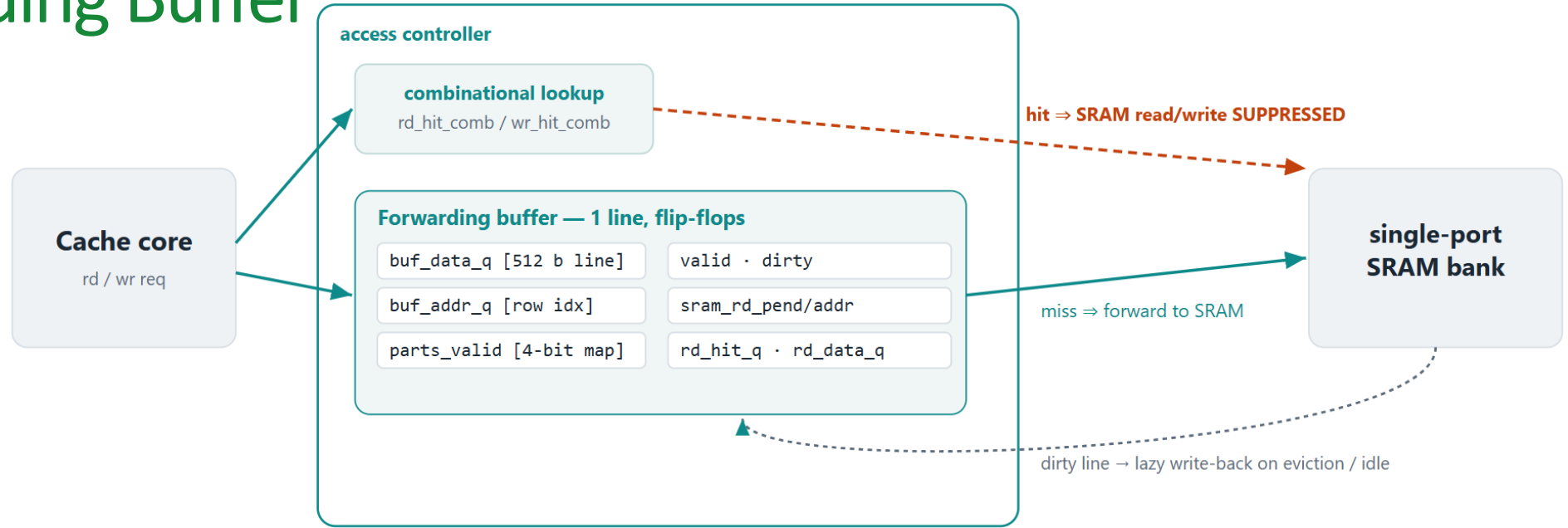
- **Folded Cache Design**

- Every 1 cycles Coalescer form a 128b visit → 1 req to **cache ctrl** from coalescer **every cycle**
- Read 512b with 4 visits from same entry
- Coalescer needs **1 cycles** to output one **128b** requests, the conflict cycle can't be hided

With folded design, the coalescer has to issue 1*128b req every cycle to sustain full bandwidth, which doesn't support by the insitu cache ctrl



Forwarding Buffer



FIELD	WIDTH (DATA PATH)	MEANING
buf_data_q	512 b (line)	the buffered SRAM row / cache line (word granularity)
buf_addr_q	~7–9 b	SRAM row index of the buffered line — not a tag
buf_valid_q	1 b	entry holds valid data
buf_dirty_q	1 b	written, differs from SRAM → needs write-back
buf_parts_valid_q	4 b (PartSplit)	which 128-b parts are actually cached



Forwarding Buffer

- Q: Solutions were: 1) LRU=>Random replacement so LRU status does not need updating 2) Dirty bit moved to registers instead of SRAM. But writing data still needs two data SRAM accesses: 1. Get complete line, 2: Write merged line. Is that correct?
- A: For data SRAM, two access: 1. get the $\frac{1}{4}$ cache line; 2. if cache hit, write new data into data SRAM, if cache miss, write the new data into data SRAM, and the byte mask into meta SRAM



Forwarding Buffer



- **Q: Biweekly31-20260421** says deterministic way selection was introduced, so only one way is read and the complete line can be read. But would this not have been possible with the initial non-folded design already, by adding this forwarding buffer and way prediction?
- **A:** That's true, we initially introduce it to reduce power consumption, but later we found that with deterministic way selection, a lot of design can be simplified. For the given case, on the performance side, it is correct, but the folded cache design can still have a lot on backend side (better area, power, routing congestion)



Forwarding Buffer



- **A more simplified design with the “deterministic way selection” is that:**
 - Deterministic way selection means each req only needs to read 1 way (instead of all ways in other design)
 - Then cache can support reading more than 128-bit for each way per cycle
 - This allows us to return to the original 256-bit req coalescer granularity
 - The coalescer can issue one request every 2 cycles to the cache pipeline
 - This potentially solves SRAM read-after-write bandwidth issues
 - However, it is still unclear whether deterministic way selection is the optimal final solution
 - For now, the RTL will retain:
 - access all ways per req
 - the existing 128-bit access path
 - the forwarding buffer
 - Further architectural exploration and performance evaluation will be conducted on the GVSoC side before committing to the final RTL design





- Q: This deterministic way selection makes the cache almost direct map. Is this correct? Did you observe any performance overhead?
- A: Yes, we use the addr hash to choose way, so it is also pseudo random replacement. And in our initial test, the performance is better than LRU, we will further investigate it later.
- Q: Have you done any sensitivity analysis on the buffer size, more than one line? Would there be a benefit? (Thinking of the case where after 4 writes you still have one stall cycle)
- A: Not yet. For now, the forwarding buffer is passive, it will only trigger a refill when there is a miss. To fully utilize multiple buffer entries, prefetching logic is needed, which will further increase the design complexity.





- **Q: Do you access both forwarding buffer and cache array in parallel? If yes, how would this help with read requests? If not, how do you deal with the performance overhead of potentially useless access of the forwarding buffer?**
- **A:** The buffer lookup is combinational (no extra read cycle, only compare the access index (5-7 bits)) and suppresses the SRAM op on a hit, the SRAM read issues only on a miss. Not “access both and drop”





- **Q:** In Biweekly 32, there was a slide with benefits of the forwarding buffer. Can you give an application example when the RAW forwarding actually happens?
- **A:** RAW = a write and a read of the same index at the same cycle, before the write goes to SRAM. The buffer serves the read from the just-written data → fresh value, no stall (vs stalling or reading stale SRAM).
- This means the RAW happens inside the cache controller pipeline, not in the processor or software programs.
- **Example:** In one cycle the core prereads request N+1 (a read) while it commits request N's write-back (a data-modifying write). If N and N+1 hit the same line, that cycle asks the single-port bank to read and write the same row.
 - The single-port SRAM can't do both, so the forwarding buffer emulates the 1R+1W, and RAW forwarding is the data-consistency piece that makes the concurrent read reflect the concurrent write.



Thank you!

Q&A





- **Q: Forwarding Buffer. Problem demonstration**

- A: See following animations on a 2 way, 2 pseudo way, 256b-cacheline origin design and 128b-cacheline folded design. Core access granularity is 128b, and refill data path is 256b.
- Animation sequence: Miss read block 0 -> trigger MSHR write to block 0, meanwhile miss read block 1
- Assuming LRU policies in both design
- Legends



Write



Read



Cycle 0:



Origin (512b data sram access per req)

Way 0 Data Bank

Pseudo
Way 0
SRAM

BLK0	BLK1
BLK4	BLK5

← 2x256b →

Pseudo
Way 1
SRAM

BLK2	BLK3
BLK6	BLK7

← 2x256b →

Way 1 Data Bank

Pseudo
Way 0
SRAM

BLK0	BLK1
BLK4	BLK5

← 2x256b →

Pseudo
Way 1
SRAM

BLK2	BLK3
BLK6	BLK7

← 2x256b →

Fold

